

Partner-hub for partner integrations (WIP)

- Owner
- Sign off
- Background Information
- Main metrics to Consider
- Consumers
- Scope and Iterations
- Component diagram
 - Special case SAP Ariba
- Existing integrations and concepts
 - DMS
 - Authentication to Connect
 - SAP Ariba
 - Authentication to Connect
 - OCI
- New Bexio integration
 - Authentication to Connect and Bexio
 - Early analysis of the Bexio flow
- API Conception
 - Inbound and Outbound models
 - REST Definition
 - API definition approval from consumers
- Migration from existing system
- Security Concept
- 12factor Assessment
- Open Questions

Owner

Historical tracking of teams owning this service

Team name	Date	Reason for change
ATLAS	May 26, 2026	Decoupling partner-interface integrations from Connect

Sign off

Before development is started we need to sign off on the service design

Name	Date
------	------

Background Information

- The partner-hub will decouple partner integrations from Connect and will be responsible to handle all communication between partner sw (such as Bexio, Ariba, DMS clients, OCI clients, etc.) and Connect, which includes authentication on both sides, token storage, and message transformation.
- Currently, Connect itself integrates all partner interfaces

i Connect provides two main integration concepts for partner sw to communicate with the webshop.

- DMS: [Decoupling partner-interface logic from Connect | DMS dealer management system the standard way of integration](#)
- OCI: [Decoupling partner-interface logic from Connect | OCI open catalog interface basic integration](#)

- Following the general OCI concept of sharing the catalog, we lately integrated SAP Ariba PunchOut: [ATLAS-2623: \[Ariba\] Lonza/SAP Integration](#) OPEN

While being similar to the general OCI idea, SAP Ariba uses a fundamentally different interface to communicate with Connect (cxml format).

Further information and documentation about DMS and OCI at [Decoupling partner-interface logic from Connect | More details about DMS & OCI](#)

- And more integrations will come for sure, each with their own flow: [Decoupling partner-interface logic from Connect | What new integrations are in progress or in the queue?](#)
- Every new integration, such as Lonza Ariba, adds new complexity into Connect, which increases the testing effort for every single release, because all the logic is hosted in the webshop itself.
[Decoupling partner-interface logic from Connect | Why are these new integrations becoming a problem?](#)
- This would also be the case with the integration of the Bexio accounting SW, where we'll have a Bexio token tied to a Connect user (not the other way around).
[bexio API documentation](#)

- We do not want to add more complexity into the Connect webshop with new partner sw integrations.

Roughly, this means that we'll create a new independent partner-hub application, outside of Connect.

- The partner-hub will communicate with Connect using the standard DMS interface.
- The partner-hub is also responsible to transform the DMS messages into messages to communicate with the partner sw we want to integrate (Bexio in this case).
- Means, connect will not contain any specific logic for Bexio. Connect will communicate with the partner-hub only, while the partner-hub will communicate with Bexio.
- Integration of the Bexio accounting SW via the new partner-hub application.

[🔗 bexio API documentation](#)

- [🔗 ATLAS-2716: \[BEXIO\] Custom DMS integration for BEXIO partner software](#) OPEN

Main metrics to Consider

- Which numbers matter in the context of your service? Could be - for example - number of orders per day, number of reviews per day, requests per minute, number of concurrent users. etc.
- Pls. consider the following example

Metric	Current Usage with Timeframe	Notes
Sessions	CH ~38000 session per month AT ~1100 session per month CZ ~70 session per month HU ~0-10 session per month	Each session triggers at least 1 Auth request 1 Open session request Optional n returnBasket request(s)
Orders	CH ~800 orders per day AT ~30 orders per day CZ ~0 orders per day HU ~0 orders per day	Each order triggers a basket transfer

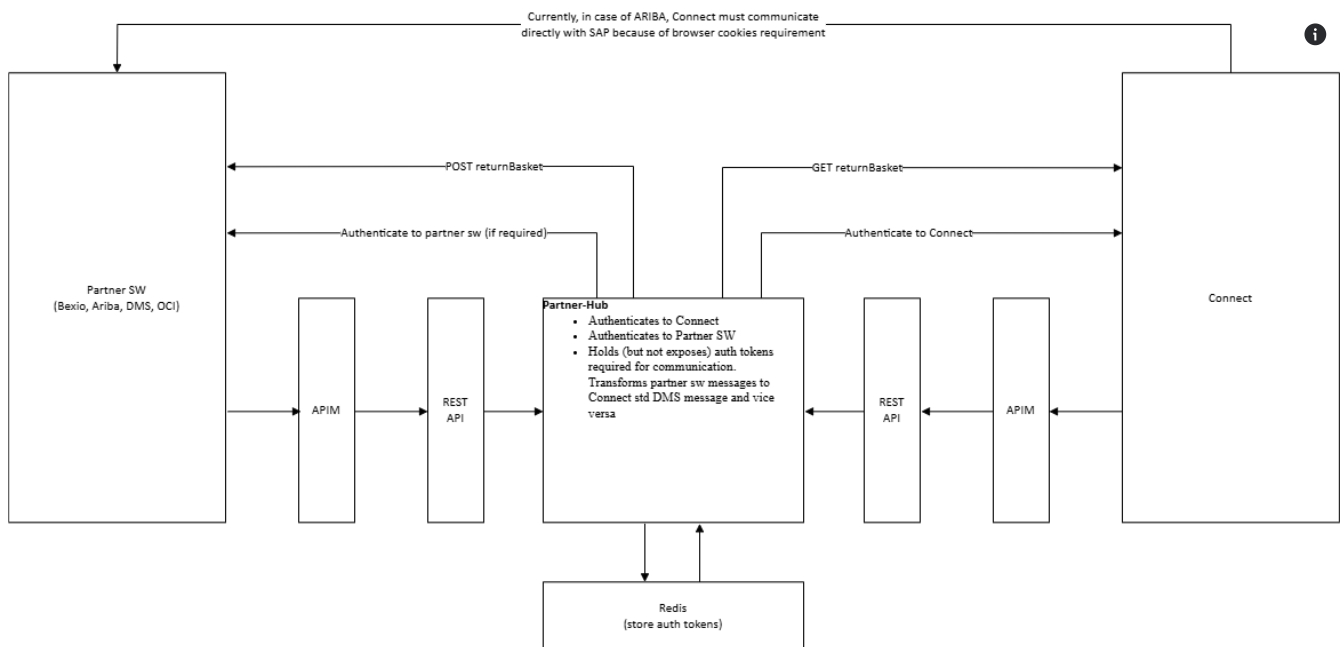
Consumers

- Partner applications like Bexio, SAP Ariba, Nextlane, DMS clients like WERK8, OCI partners (usually SAP, e.g. used by SwissArmy).
- Connect FE and BE for basket and order transfer

Scope and Iterations

- Bexio will be the pilot integration for the new partner-hub.
- Later integrations like Nextlane will follow
- Furthermore, at some point in the future, we'll move existing integrations out of Connect and into the partner hub.
- One greater goal of the partner-hub is to 1) stop adding more complexity due to more partner integrations in Connect, and 2) to take existing complexity out of Connect.
- Furthermore, the initial version or phase of the partner-hub will be our own implementation. At a later stage, the message transformations will be done by Boomi (but that's out of scope for this project)

Component diagram



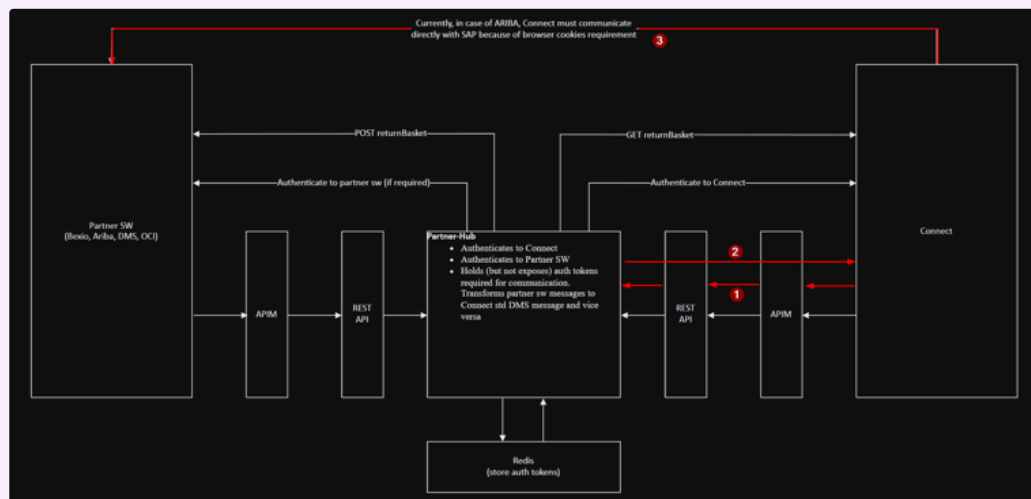
Special case SAP Ariba

In Case of SAP Ariba, it is a requirement that the frontend, in our case the browser that runs Connect, must fire the request to send the basket data back to SAP. Means it cannot be sent by the Partner Hub on behalf of Connect. This is required because the Connect

runs within an iFrame, and the sender (browser) must have access to a cookie provided by the parent html.

To still decouple the creation of the cxml PunchoutOrderRequest from Connect, we'll have another interface **(1)** on the PartnerHub which lets Connect request the required request body, in this case cxml for SAP Ariba. The PartnerHub will then return the particular request body as response **(2)**.

The Connect frontend will then send the actual request **(3)** using the given request body to the partner system (SAP Ariba).



Note that the above just explains what is needed as a result.

How this can be technically done in a more elegant way as the one explained, is a different story, which can be discussed at a later stage.

Existing integrations and concepts

DMS

See [Decoupling partner-interface logic from Connect | DMS dealer management system the standard way of integration](#)

Authentication to Connect

Currently integrated DMS and SAP ARIBA clients only authenticate to Connect using the visit registration service in Connect.

 20241007_DMS-Connect-Visit Registration-Service-a.pdf



Standard DMS clients authenticate on their own.

20260422_DMS-API-version-5.1.pdf

SAP Ariba

Authentication to Connect

SAP Ariba authenticates via AZURE APIM, where APIM triggers the visit registration service.

More details about SAP Ariba at:

- Overall documentation [🔗 ATLAS-2623: \[Ariba\] Lonza/SAP Integration](#) OPEN
- Basket transfer using cxml [🔖 ATLAS-2730: \[Ariba\] Transfer order using Ariba cxml format. Manually placing order](#) DONE

OCI

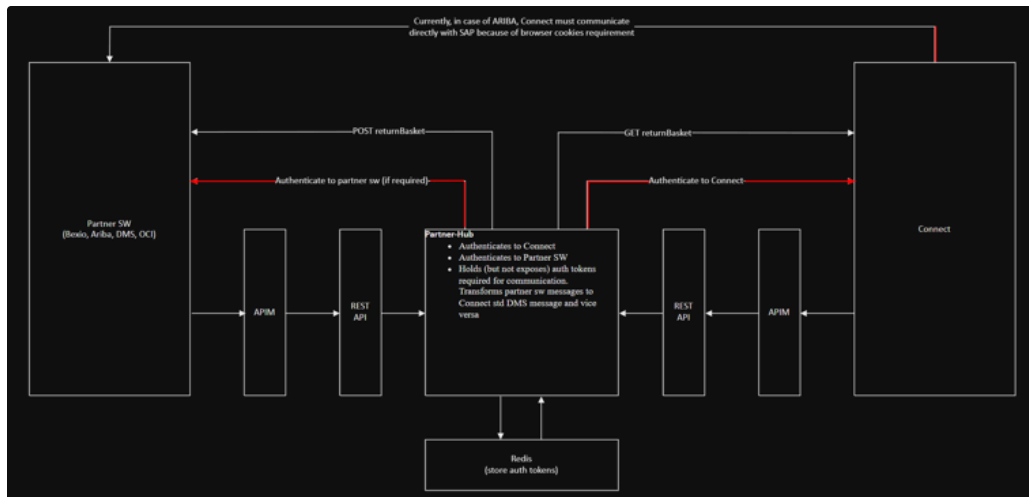
See [Decoupling partner-interface logic from Connect](#) | [OCI](#) [open catalog interface](#) [basic integration](#)

New Bexio integration

Authentication to Connect and Bexio

Other than standard DMS clients, Bexio requires Connect to authenticate to the Bexio API.

The partner-hub will authenticate to both Bexio and Connect, and will store the authentication tokens via Redis in memory (and optionally in a DB).



Early analysis of the Bexio flow

(for further information, refer to the Bexio API documentation at [Authorization flow - bexio API documentation](#) and [Refresh token flow - bexio API documentation](#))

Bexio is a general mini SAP like system, which is not specifically targeting garages. Its target audience are partners of all business segments (Bank institutes, Insurance, Treuhand, any kind of KMUs, etc.).

This also explains, why they do not want to use our DMS interface as is.

While our DMS integration requires the partner application to authenticate with Connect, the Bexio application requires us (Connect) to authenticate with their API.

When a D-Store Customer uses the Bexio link to Connect for the first time, then the authentication with Bexio is required.

1. The user opens Connect (D-Store) using a bexio link
 - a. The user clicks Login within "Connect (D-Store)".

There are 3 options to authenticate in D-Store.

The problem with 1) and 2) is that Bexio uses the email address as unique identifier to map the Bexio user to the D-Store user.

On the other hand, the email address in D-Store is not unique, not even within the affiliate.

This means, even when using the 1) or 2) flow, the Bexio users would still have to manually log in to D-Store every time they use D-Store from Bexio.

The only way to avoid this would be that Bexio would allow to store the unique D-Store username together with the Bexio user account.

With the D-Store username we could map the incoming init session request to the correct customer and user in D-Store

i. Fully automatic

- a. D-Store allows Bexio to act as an authentication provider
- b. Bexio issued the authentication token on first Connect login
- c. Connect stores the token and users it for the upcoming logins
- d. Bexio sends the user's email address to map with the D-Store user

ii. Semi automatic

- a. The user manually creates an auth token in Bexio,
- b. On first Login in Connect, the user pastes the token into a new UI component provided by D-Store.
- c. Connect stores the token and users it for the upcoming logins
- d. Bexio sends the user's email address to map with the D-Store user

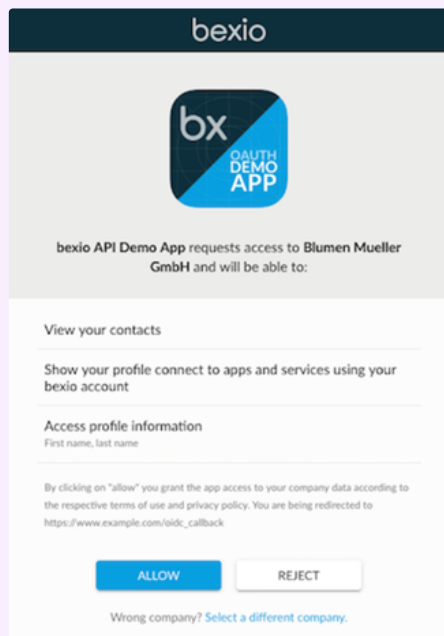
iii. Fully manual

- a. Manual login is always required on both sides, Bexio and D-Store.

2. The **Connect (D-Store)** redirects the user to the /authorize endpoint of the bexio OpenID Connect service.

3. The bexio OpenID Connect Service displays the login page.

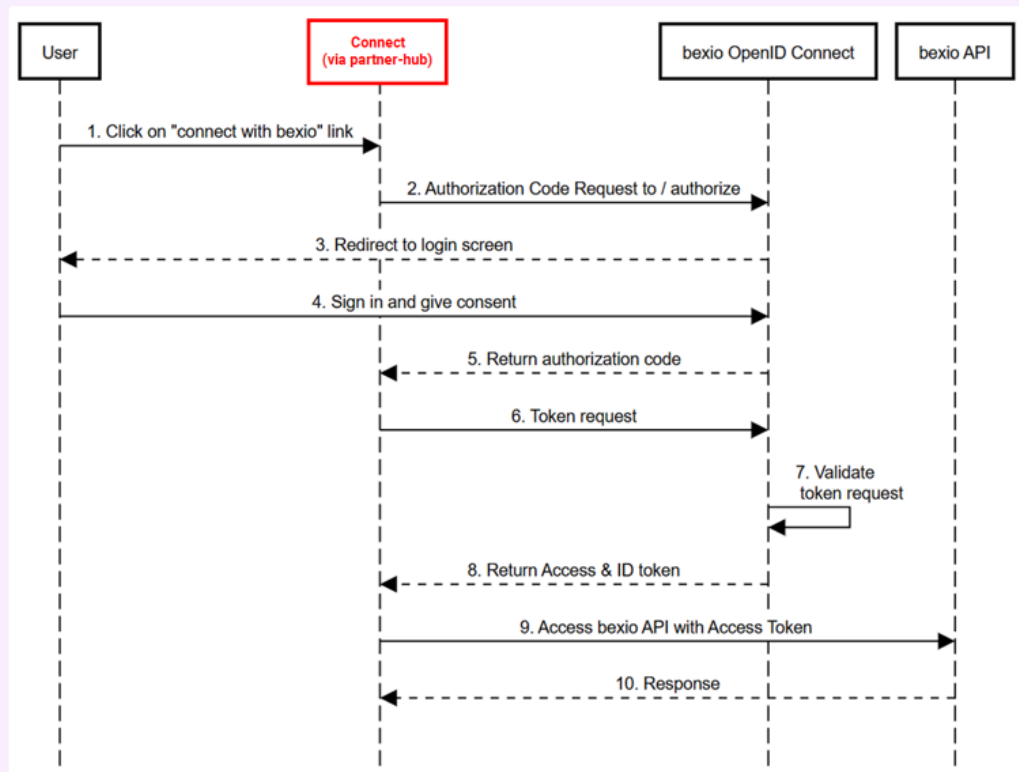
The user authenticates and sees a consent page listing the permissions bexio will give to the **Connect (D-Store)**.



5. The user is redirected back to the **Connect (D-Store)** with an Authorization Code.

6. The **Connect (D-Store)** sends this code to the bexio OpenID Connect service (/token endpoint) along with the application's Client ID and Client Secret.

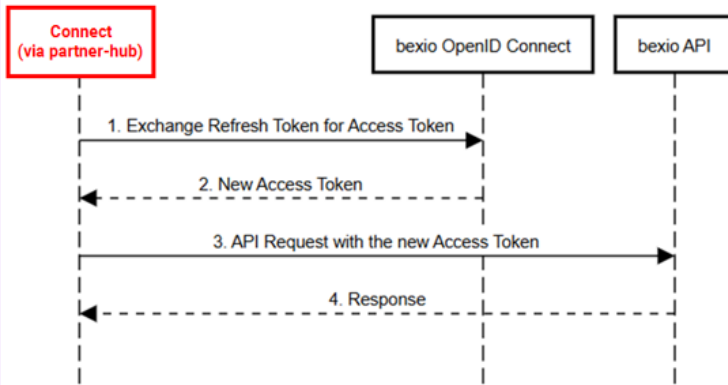
- a. From what I understand, the Client ID and Client Secret is something Bexio will provide to us.
7. bexio verifies the code, Client ID, and Client Secret.
8. An ID Token and Access Token (and optionally, a Refresh Token) is returned to the **Connect (D-Store)**.
9. The **Connect (D-Store)** uses the Access Token during the session to call the bexio API.
10. The bexio API responds with requested data.



The scope `offline_access` is required to obtain a refresh token to keep the api connection alive.

Means, once a customer is authenticated on Bexio's side, a refresh token can be stored on in Connect on user level to re-authenticate (refresh the authentication) without having to login again on Bexio side.

1. POST a request to the endpoint `"/token"`. Make sure you use the grant type `"refresh_token"`.
2. The response contains a new access token. Please note that the requested scopes do not change when refreshing a token. Acquiring new scopes is only possible by going through the initial authorization process again.
3. The new access token can be used to authorize requests and execute API requests
4. The bexio API responds with the requested data.



Conclusion:

Bexio will be an entirely new integration. Though, we can reuse some concepts of DMS, OCI, and ARIBA.

- What we can re-use the general handling of storing offers, preparing return baskets, etc.
- Also, the Connect authentication flow can be reused.

Entirely new implementation

- The partner hub authentication against Bexio will be entirely new.
- Also, the basket data transfer back to Bexio will be entirely new.

For further details, see the Bexio docs at [bexio API documentation](#)

API Conception

- Independent application, outside of Connect
- General function of the partner hub is to transform
 - Standard DMS messages coming from Connect into partner sw specific messages to partner sw
 - Partner sw messages coming from partner sw into standard DMS messages to Connect
- Message flows (REST API)
 - Generic Authentication flow
 - Connect $\Leftarrow$ APIM $\Leftarrow$ Partner hub $\Leftarrow$ APIM $\Leftarrow$ partner sw $\Leftarrow$ Customer
 - Bexio specific authentication flow (see detailed flow below or in the docs at [Authorization flow - bexio API documentation](#))
 - Connect $\Leftarrow$ ~~APIM~~ $\Leftarrow$ Partner hub $\Leftarrow$ APIM $\Leftarrow$ partner sw (Bexio) $\Leftarrow$ Customer

=====⇒

=====⇒ (e.g, std OCI or Ariba Punchout case)

- REST API for the POST returnBasket flow, letting Connect transfer the basket to the partner hub
 - Customer ⇒ Connect ⇒ APIM ⇒ Partner hub ⇒ partner sw (Bexio in this case)
- REST API for the GET returnBasket flow, letting the partner SW
 - Connect ⇐ ~~APIM~~ ⇐ Partner hub ⇐ APIM ⇐ partner sw ⇐ Customer
- Capsulation of authentication tokens in the partner hub
 - The partner hub authenticates against involved systems (Connect and partner sw), stores the tokens, but never exposes them to Connect or the partner sw.
 - Authentication flows
 - Partner hub authenticates to Connect ⇒ Stores auth token in memory using Redis
 - Partner hub authenticates to partner sw (in case of Bexio) ⇒ Stores auth token in memory using Redis
 - returnBasket flows
 - POST returnBasket flow ⇒ Partner hub uses the partner sw auth token (if required) to communicate with the partner sw API
 - GET returnBasket flow ⇒ Partner hub uses the Connect auth token to communicate with Connect
 - Technical setup
 - REST API: AZUR function app (preferred over always running docker container based service)
 - Dedicated DB: AZURE SQL

Inbound and Outbound models

- For the “reachable” APIs document:
 - the inbound data model, i.e. what goes in?
 - the outbound data model, i.e. what goes out?
- Make sure to any type of input/output that is exposed to consumers:
 - REST models
 - Event models
 - ...

REST Definition

- Include definition for exposed endpoints
 - Parameters

- Name
- Type
- Required
- Error codes it can return
 - How to handle the error codes
- Limitations / Rate limits

API definition approval from consumers

Service Name	Team	Concerns/Missing functionality	Vote
Partner-hub	ATLAS	Due to browser cookie requirements in case of SAP Ariba, Connect must call the partner API directly. Analysis ongoing: possibility to transfer of browser cookies to the backend  ATLAS-2886: Investigate the possibility to transfer the browser cookie from FE to BE in Ariba basket transmission process 	 

 Service Definition note complete without a thumbs up from all consumers

Migration from existing system

- Migration is not required for the initial implementation
- Stored offers remain in the Connect DB. The partner-hub will just transform the messages:
 - Partner sw $\iff$ Connect

Security Concept

- Who has access to the service from our internal groups?
- Which authentication mechanisms are used to communicate between services?

- How do we safeguard that no unwanted access can occur?
- What's the level of risk involved (higher risk for customer data, purchase prices; lower risk for data that's publicly available like product data that can be found on the webshop)?
- Which authentication mechanisms are used for public access (e.g. JWT tokens via APIM)?
- Where are the secrets stored (only possible answer: keyvault)?
- How can we detect unwanted access or intrusions?
- How do we continuously monitor the service's security guidelines are in place.

12factor Assessment

Add the assessment table below after the implementation of the phase is complete. If multiple phases exist update this table after every phase

See  [Service completion process](#)

Open Questions

- How should it be set up country-wise?
 - One partern-hub service per country?
 - Or one for all countries(like the SWS middleware is done)?.
 - Generally, the Bexio integration will only come for CH. Furthermore, SAP Ariba is also used in CH alone. Therefore, we could start with CH alone for the moment.